Publisher-Grounded Discovery for an Open Agent Tool Web

OpenBot

***Abstract*—An open web of agent tools needs discovery without making one registry the gate through which all publishers must pass. A public name is not proof of quality, a failed fetch is not a withdrawal, and public search must not carry private account data. This design and analysis paper defines a publication profile based on stable Card URLs, evidence from publisher documents, and separate public entries and private connections. It records each evidence path so that a missing declaration can be told apart from a path that could not be checked. Search across several indexes retains failed-source and document-conflict information. Fully specified baselines expose the errors caused by mixing negative and unknown evidence or keeping only the first returned description. A possible-world model gives the strongest publication decision justified by a fixed set of partial observations. Two counterexamples bound the claim: permanent outages prevent a finite withdrawal guarantee, and separately read documents need not describe one publisher state. Finite executable checks test the state rules and comparison policies. They do not establish field performance or product conformance. A proposed network study covers source failures, withdrawal, and endpoint changes.**

***Index Terms*—Agent tools, Service discovery, Publisher evidence, Public indexing**

I. Introduction

An agent host must find tools before offering a connection. A single registry makes discovery simple, but makes registration and registry availability part of publication. A publisher already controls an HTTPS address. The question is how to build public search from that address while keeping publication intent, source identity, and uncertainty clear.

A Card describes where a service can be reached. A Catalog lists Cards. An index keeps public descriptions for search. A private connection holds user settings and sign-in state. These change on different schedules. A publisher may end public listing while existing users still need the service. A Catalog outage need not change publication intent. Two indexes may hold different Card versions. A service may keep its title while changing the endpoint that receives user data.

One trust bit cannot express these cases. Deleting after every failed check turns a network fault into a false statement about the publisher. Keeping every entry forever makes withdrawal ineffective. The design must preserve uncertainty while acting on clear evidence.

We define a publisher discovery profile, meaning a set of rules for using existing web document formats. The profile uses a Card URL as service identity, requires anonymous publication checks, and joins independent indexes while retaining failed-source information. The profile is a proposed design. Its rules, comparison baselines, proofs, and finite executable checks are given here. The checks are separate from the product and from a live web deployment.

The contributions are a model separating identity, intent, visibility, and connections; a precise decision rule for mixed removal and outage; proofs that state its information and time limits; and finite comparison checks with a plan for network validation. The scope is publication and discovery across independent indexes. Protocol conformance generation and tool invocation decisions are outside it.

II. Standards and related designs

RFC 3986 describes URI syntax, resolution, and comparison. Our profile uses a parsed HTTPS URL as its identity key and keeps the path and query. Matching names or descriptions cannot merge distinct keys. The normalization rule below makes this choice reproducible. [1]

RFC 8615 gives the well-known path space used for origin-level discovery. RFC 8288 provides a model for typed links and their form in HTTP headers. Together they support two useful entry routes: a domain can lead to a fixed discovery location, while a page or response can lead to a more specific Catalog. Neither document makes a discovered service safe to use. The publication policy in this paper is a layer above those location and link mechanisms. [2], [3]

The AI Catalog draft describes a typed list of artifacts that can point to native descriptions. It supports richer identity and trust material, but explicitly warns that identifier syntax does not establish trust. Our profile selects publisher-linked Catalog entries that name an exact Card URL. Optional trust material is outside the profile; its presence does not change the rules below. [4]

The experimental Server Card proposal hosted by the MCP organization describes metadata for discovering a server before a live protocol exchange. Its README states that the work has not been accepted as an official MCP extension and is for prototypes and feedback. The discovery document labels its revision as draft, and the README calls the v1 shape pre-release. The discovery text also separates a Card's claims from live behavior and says that Card content must not decide security or access control. These limits support our separation between a source claim and a private connection review. [5], [6]

The worked profile selects Catalog documents marked `1.0`, the experimental Card v1 shape, and endpoints advertising MCP `2026-07-28`. The dated MCP specification is a primary reference for that protocol revision. This selection does not make the separate Card proposal or Catalog draft a final

standard. Other revision profiles can reuse the state method if they preserve its evidence meanings. [7]

Crawler rules answer whether a crawler may read a resource. RFC 9309 defines robots rules, matching, redirects, and failure handling. It expressly separates these rules from access authorization. Google documents `X-Robots-Tag` as a way to control search indexing, including `noindex`. Our proposed profile adopts a Card-level `noindex` rule. This is a defined publication policy, distinct from the robots standard and from Google's search behavior. [8], [9]

HTTP validators allow a crawler to ask whether a stored representation changed. They reduce repeated transfer, but do not prove that every related Catalog remains unchanged. A valid unchanged Card can coexist with a changed publication declaration. Thus conditional requests are part of evidence collection, not a replacement for the publication decision. [10]

WebFinger is an earlier HTTPS method for finding links and properties about a named resource. It permits different public and private responses, and its security section separates transport protection from the truth of published claims. We share these limits. This profile adds a life-cycle rule for retained public entries when several publisher paths give partial or conflicting evidence. It does not replace WebFinger or claim that domain-based discovery is new. [11]

Chandra and Toueg separate failure detector accuracy from completeness in systems where delayed messages can resemble failed processes. Here the hidden fact is a publisher declaration, not a process crash. The same need to separate lack of response from proof motivates our unknown result. Our local impossibility argument below is about discovery decisions; it is not a consensus result or a new general failure detector. [12]

Green, Karvounarakis, and Tannen describe how source annotations can be carried through database queries. Our path and source records are a limited form of this wider provenance idea: provenance means information about where a result came from. We use sets of source records to expose disagreements. We do not introduce a new provenance algebra, infer independence from source count, or compute a probability of truth. [13]

The contribution is the combination of a publisher-path policy with explicit unknown outcomes, retained path history, and separate connection review. An evidence path has a known root, observed documents, and an explicit result. The finite results concern the defined policies only. They are not a claim of superiority over all registries or discovery protocols.

III. System model and threats

The system has publishers, hosts, users, and index operators. A publisher controls a Card at URL c. The Card may point to a service on a different host. A host reads documents and may keep a private connection. An index operator stores public metadata and accepts discovery hints. Operators can share search results, but they need not share storage, policies, or accounts.

Let a public entry be (c, d, E, t), where d is the Card, E is the set of accepted publisher evidence paths, and t is the last successful check time. Each path records an association type, its publisher location, its Catalog URL, and the exact Card URL that was listed. A private connection is a different record, keyed by user and connection identity. It includes configuration and sign-in state. Public entries have no field that requires those values.

We assume that the public network adapter checks HTTPS targets, rejects private network destinations, and enforces document and time bounds at each redirect. We assume that stored checks are applied by one current job owner. HTTPS gives evidence about the responding origin under the network trust model. It does not prove the legal identity, honesty, competence, or future behavior of its operator. A stolen domain or server key falls outside this origin-control assumption.

Threats arise even under these assumptions. A third party can copy a respected service name into its own Catalog. A publisher can place hostile instructions in a description. A response can redirect a crawler toward a private address. An operator can keep an old Card after withdrawal. Several sources can repeat the same false claim. A previously sound publisher can change the service endpoint. Ordinary outages create similar observations to deliberate removal unless the method keeps the difference.

TABLE I
Evidence has a limited meaning. None of these facts alone permits a tool call.

Observed fact	Permitted conclusion
Same Card URL	Same service key under the URL policy.
Publisher Catalog lists Card	An accepted source path declares publication.
DNS or Card token matches	The claimant controls the stated proof scope.
Many indexes return Card	Several sources report it; quality remains unknown.
Card fetch fails	The current Card is unconfirmed.

The desired result is an explainable public listing, not a certificate of safety. The index must preserve source locations even when names are equal. It must treat descriptions as data. It must honor clear withdrawal signals without taking an outage as one. It must also expose partial search failure, so a user or agent can distinguish an empty complete response from an incomplete view.

IV. A publication method based on evidence paths

A. Discovery and identity

The profile accepts a domain, a service page, a Catalog URL, or a Card URL. A domain selects `/.well-known/ai-catalog.json` at its HTTPS origin. A page names a Catalog through a static `ai-catalog` link or an HTTP Link header. Scripts do not run. Catalog entries with Card media type and a URL become candidates; inline-only entries cannot establish the stable URL required here. Finding several candidates does not connect any of them.

A candidate is only a location hint. To enter the public index, the Card is read again through the anonymous public

adapter. The indexer does not reuse a private authenticated response. It checks document shape and a supported endpoint before considering publication evidence. A compatible endpoint is a metadata condition; live capabilities remain a separate connection concern.

The profile rejects non-HTTPS URLs, user information, and fragments. Its key lowercases scheme and host, removes an explicit port 443, removes dot segments, and gives an empty path `/`. It preserves path case, query order, and other percent encoding. Relative links are resolved before this step. This conservative equality rule may keep equivalent spellings separate. Redirects remain observations and do not merge identities at their final location. A move to a new Card key creates a distinct service identity under this profile.

B. Establishing a publisher path

Three paths establish association: the fetched Card origin's well-known Catalog; a Catalog link on the Card response; or a Catalog linked from the page named in the Card's website field. A path key contains its association type, root location, and Catalog URL. Each accepted Catalog must list the exact normalized Card key. An unrelated Catalog found through search supplies a hint, never publisher evidence by itself.

This rule supports hosted services. A Card served by a hosting company can link to a customer's Catalog, and that Catalog can link back to the Card. The evidence shows this observed relation. It does not prove that the hosting company and customer are one legal entity. The source path must remain visible, so a consumer can see which party made which declaration.

The method stores an observation for every previously accepted path. Positive means a currently accepted path lists the Card. Negative means a valid fetched Catalog omits it, or a fresh and completely parsed root removes the link that established the path. Unknown covers network failure, denied Catalog reads, invalid documents, and exhausted budgets. A Catalog response of 404 or 410 is unknown under this profile: absence of the document does not state which service declaration was withdrawn. Missing responses never become empty Catalogs.

Each check starts prior path results as unknown, then fills them from fresh reads or validated cached responses. If no path is positive, withdrawal needs every prior path to be negative. A prior path set must be nonempty. Evidence about an empty set cannot establish withdrawal. On saving an indexed Card, retain positive and unresolved prior paths and add new positives. Thus one successful path cannot erase another's uncertain history. This decision concerns known evidence, not the whole web. Each observation carries its read time. Separate reads do not form an atomic snapshot, meaning a view of all documents at one instant. The proof against false withdrawal therefore assumes that the relevant declarations stay fixed during the check; the time-varying case is treated below.

C. Separating state dimensions

A state has three parts: a retained public record, search visibility, and the latest check result. They are separate because a crawler may retain old bytes while hiding them, or display a previous record while reporting an unconfirmed refresh. A deletion flag alone cannot represent both cases.

Fresh Card-level `noindex` removes the public listing. A robots denial of the Card read hides the entry through an excluded check; it does not prove withdrawal. Robots fetch failure prevents the read and makes the check unconfirmed. With a valid Card, positive evidence gives indexed status. All prior paths being negative gives withdrawn status. Otherwise uncertainty preserves the prior record and visibility. A first candidate without evidence stays hidden, with excluded status after complete checks or unconfirmed status after incomplete checks.

TABLE II
PROPOSED TRANSITIONS. WITHDRAWAL ASSUMES AN EARLIER ENTRY; OTHERWISE THE RESULT IS EXCLUSION.

Current observation	Check	Public effect
Valid Card and positive path	Indexed	Save and show.
Fresh Card says noindex	Withdrawn	Remove listing.
Robots denies Card read	Excluded	Hide; keep reason.
All prior paths negative	Withdrawn	Remove listing.
A needed path is unknown	Unconfirmed	Keep prior state.
First complete check lacks evidence	Excluded	Do not add.

Keeping prior visibility can leave stale metadata in search. The response must include both the last successful check time and the latest failed attempt. A deployment may apply a separate age limit that hides very old entries. Such hiding is a host freshness rule, not evidence that the publisher withdrew. The rule and reason should remain distinct in storage and responses.

D. Applying checks without late writes

A background check receives an owner token and a job generation. It gathers observations outside the final write transaction. Before changing the entry, the transaction verifies that the token still owns the job. If another worker has claimed it, the old result is discarded. The check result and public change then become one accepted job outcome.

This guards against a simple race: a slow positive check finishes after a newer check saw `noindex`. It does not create one global order across independent indexes. It also does not make duplicate network reads impossible. The useful property is narrower: a worker that has lost ownership cannot apply its stale result to that job.

E. Search across independent indexes

Each search records the selected source set, successful sources, and failed sources. Results are merged by Card URL. Matching titles at different URLs remain separate. Each result includes its reporting sources. One failed source produces partial results with issues; failure of every selected source produces an unavailable response.

Define a first-wins search baseline as follows. Visit successful sources in configured order. For a new Card key, keep its body and check time. For a repeated key, add the source label

but discard its body and time. Our proposed merge instead retains each source's body digest and check time. A digest is a short value derived from the document bytes. Different digests indicate disagreement, although harmless formatting can cause it. Digest equality uses a collision-resistant hash as an engineering assumption; a complete byte comparison can remove that assumption. It still does not prove that the common body is true. Source count does not settle which description is current. A bounded publisher recheck may itself remain unconfirmed.

The profile sorts results by normalized Card key. This avoids choosing a name when source reports disagree. A display can show all reported names with their sources. Its page cursor holds the last Card key, the query, selected sources, and unavailable source set. A mismatch in query or source conditions rejects continuation and requests a restart. This avoids silently changing the failure view. It does not freeze source contents. A stable snapshot would need versioned source results or a saved result set.

F. Private connections and changes

The index worker receives only a public Card URL and public hints. It cannot read user secrets through its service interface. Removing an index entry does not delete a user's connection, and an unavailable index does not prevent a known connection from being used under its own rules. Conversely, a private sign-in is not evidence that a Card is eligible for public search.

A connection comparison projects each endpoint onto its URL, transport, supported version set, header names, and input fields. Input fields include format, secret flag, required flag, default, fixed value, and allowed choices. Maps compare by key and version sets by membership. Text and unrelated extensions are excluded. Any change in this projection raises a review signal; it never moves saved credentials automatically. Publisher proof and index inclusion cannot waive review. The later authorization decision is outside this paper.

V. Defined publication baselines

All comparison policies use the same fetched observations, Card checks, `noindex` priority, and Card-level robots exclusion. They differ only in admission and evidence aggregation. These are defined algorithms for analysis; no deployment is claimed.

The **mention baseline** indexes a valid Card if any fetched Catalog lists it, including an unrelated Catalog; otherwise it removes an existing listing. Failed reads count as absence. The **collapsed baseline** requires a publisher path, but treats negative and unknown path results alike. It removes an existing listing whenever no path is positive. Both keep a first candidate hidden until admission passes.

The **mixed-evidence baseline** requires a publisher path. If any path is positive, it indexes the Card. Otherwise, if any prior path is negative, it withdraws an existing entry. Otherwise it returns unconfirmed when any needed check is unknown. A complete first check without evidence returns excluded. Its distinguishing rule is that a negative path takes priority over unknown paths when no positive path exists.

The proposed method instead requires every prior path to be negative in this check. In the procedure, `old.paths` is nonempty for an existing listing. `unknown` includes incomplete Card validation, incomplete first-time path search, and unresolved paths. Before entry, a blocked Card read returns excluded; robots fetch failure returns unconfirmed. The `Negative` predicate reads each path's result for this check.

```
decide(old, check):
  if check.freshNoindex:
    return removeOrExclude(old)
  if not check.validCard:
    return keep(old, Unconfirmed)
  if check.hasPositivePath:
    return save(check.card, retainedPaths, Indexed)
  if old.exists and all(old.paths, Negative):
    return remove(old, Withdrawn)
  if check.unknown:
    return keep(old, Unconfirmed)
  return hide(old, Excluded)
```

`keep` leaves bytes and visibility unchanged; it changes only the latest check. `save` records fresh metadata, shows it, and retains positive and unresolved prior paths. `remove` deletes public listing bytes and keeps a withdrawal record. `hide` preserves any stored bytes while excluding them from search. No operation changes a private connection. The first-wins search baseline is an independent choice and can be paired with each publication policy.

VI. Worked cases

A. A copied name and a hosted publisher

Suppose `studio.example/card` and `copy.example/card` both have the title "Schedule Helper." A third-party Catalog lists both. Discovery returns two candidates. It cannot merge them by title, and the Catalog itself proves neither publisher relation. If the first Card's own origin lists it, that candidate can be indexed. The second must pass its own publisher check.

Now suppose the first Card is at `host.example/studio/card`. Its response links to a Catalog at `studio.example/apps`, and that Catalog lists the exact hosted Card URL. This is an accepted Card-link path. A future response can still change or remove it. Search should show both the Card location and the association source. The phrase "publisher evidence" therefore describes a chain of public statements, not an endorsement of the service.

B. Removal mixed with an outage

At time t_0, a Card has two accepted paths, a and b. At t_1, the fetched Catalog for a no longer lists it. The Catalog for b times out. The observation vector is `(negative, unknown)`. Under the proposed method the check is unconfirmed and the earlier listing stays. The host has evidence that one path ended, but lacks evidence about the other.

At t_2, path b returns a valid Catalog that still lists the Card. The vector is `(negative, positive)`, so the listing is current again with one path. If it instead returns a valid omission, the vector is `(negative, negative)`, and withdrawal follows. A Card response carrying `noindex`

would remove the listing directly, without waiting for both Catalogs.

The mixed-evidence baseline takes its negative-path branch at t_1: no path is positive and a is negative. It therefore withdraws. The collapsed baseline also withdraws. Under the stated fixture, path b still declares publication during its outage. Both baselines produce a false withdrawal relative to that fact. The proposed method preserves the listing because its all-negative condition is false. This counterexample follows from the rules and does not need measured failure rates.

C. Partial search and an endpoint move

Consider three illustrative sources. Source A returns Cards 1 and 2. Source B returns Cards 2 and 3. Source C fails. The merge contains three distinct Cards because $2 + 2 - 1 = 3$. Card 2 has two source labels. The fraction of responding configured sources is $\frac{2}{3}$, but this is not search recall: source C might contain no extra Cards or thousands of unique ones. The failure must therefore remain visible as an issue.

If C recovers before the next page, the changed failure set invalidates the cursor. Suppose A reports Card 2′s endpoint as `/old` and B reports `/new`. First-wins picks `/old` in order A,B and `/new` in order B,A, while showing the same two source labels. Thus source ordering changes the chosen body without showing the conflict. The proposed merge preserves both digests and times. A fresh publisher check may resolve the description, but cannot transfer a connection's credentials to a new endpoint without review.

VII. Analytical results and limits of the claims

A. What a partial observation can decide

Fix a nonempty set of prior paths $E = \{1, ..., n\}$. For this result only, hold each path declaration fixed throughout one check. Let $x_i \in \{0, 1\}$ say whether path i declares the Card. Define $P(x) = 1$ exactly when some $x_i = 1$. This definition does not refer to a publisher's hidden intention or an undiscovered path. Let $o_i \in \{+, -, ?\}$ be its observation. A sound classifier reports $+$ only if $x_i = 1$, and $-$ only if $x_i = 0$. A failed or incomplete read reports ?.

Define the compatible worlds $W(o)$ as all binary vectors that agree with every known observation. For an unknown coordinate, both values are allowed. No independence or probability model is assumed. Every statement about publication must hold in all worlds in this set if it is to follow from this evidence alone.

This is an instance of established incomplete-information semantics: a certain answer holds in every completion allowed by the observations [14]. We specialize that rule to the disjunction of publication paths. The proposition is not a new general result about three-valued logic.

Proposition 1: strongest certain decision. Publication is true in every world in $W(o)$ exactly when some coordinate is positive. Publication is false in every world exactly when every coordinate is negative. In every other case, both a published and an unpublished world are compatible with the observations.

Proof. A positive coordinate makes the disjunction true under every completion. If no coordinate is positive, fill all unknown coordinates with zero to obtain an unpublished world. All-negative observations have only that zero vector as a completion. Otherwise there is an unknown coordinate. Set one such coordinate to one and the others to zero to obtain a published world. These cases cover every vector. Thus a rule that makes more certain publication decisions from the same observations must be wrong in at least one compatible world. The result is information maximality under this model, not a claim that preserving a listing always gives the lowest social or operating cost.

The implemented decision model keeps a prior listing in the undecided case. An age-based host policy may instead hide it, but must name age rather than confirmed withdrawal as the reason. A first candidate is different: uncertainty supplies no positive evidence for admission. The result also explains why adding more negative paths cannot settle the matter while one prior path remains unknown.

B. Safety, time, and independent sources

No name collision merge. Assume that parsing yields stable Card keys and the merge map is keyed only by those keys. Two entries with distinct keys cannot become one entry merely because their text fields match. The insertion operation only combines values at an equal key. By induction over the input sequence, the final map preserves the two identities. This result does not establish that either source is honest.

No outage-only withdrawal. Assume the proposed per-path rule, an existing entry, no fresh `noindex`, no Card-level robots denial, and no independent host hiding rule. If a check contains only unknown observations about prior paths, the all-negative condition is false. The rule preserves the prior public state and records uncertainty. This proof depends on the observation classifier: a timeout misclassified as a valid omission would break it. The stronger mixed-evidence trace has the same result while any prior path remains unknown and no path is positive.

Private-value independence. Hold the discovery seeds, public responses, clock values, and host policy fixed. Change only the values stored in private connections. If public indexing has no access to that store and its input contains no private fields, its public output is unchanged. Each step reads the same inputs and applies the same rules. This is a data-flow claim, not total privacy. A connection event can reveal timing by causing a public crawl, and a publisher can itself put sensitive text in a public Card.

A limit under permanent outage. Suppose a listed Card has one prior path. Compare two executions after time t. In A the path continues to declare the Card. In B it removes the declaration. Every request after t times out in both executions, and every other observed input is the same. A deterministic checker receives the same history and must make the same decision in both. If it always preserves visibility in A, it cannot guarantee removal by any finite deadline in B. For a randomized checker, couple the same random choices in both runs and the argument applies to each choice sequence. Thus outage-only preservation and a finite withdrawal deadline

cannot both be guaranteed without a new assumption. An expiry policy, a trusted push notice, or eventual successful reads changes the assumptions; none is a consequence of a timeout.

Non-atomic read counterexample. Two paths can cause a different error even with no failed request. Initially a is absent and b declares the Card. The checker reads a and records negative. Before it reads b, the publisher adds a, then removes b. The checker reads b and records negative. Some path declared the Card at every instant, but the assembled vector is all negative. Therefore sound individual reads do not prove a state that held at a common instant. Proposition 1 assumes fixed declarations across the check. In a live service, a shared publisher generation and a validated snapshot could supply a stronger contract; without one, the withdrawal reason means that all known paths were observed absent during a check interval. A short interval reduces exposure to changes but is not a proof. The same limit applies within a root-and-Catalog evidence path.

A local withdrawal bound. Assume a publisher serves `noindex` continuously after time t. A check is scheduled within D time units and dispatched within a further Q. Assume that this dispatched check successfully reads a fresh `noindex` response and commits its accepted removal within C of dispatch. Its dispatch time s then satisfies $s \leq t + D + Q$, and removal occurs by $s + C \leq t + D + Q + C$. This is a conditional liveness claim: removal happens within the bound only under the stated success assumption. A bound on the duration of successful checks alone does not ensure success. If all reads time out or all results lose job ownership, the listing may stay without a finite removal bound. The earlier safety claim against outage-only withdrawal needs no eventual successful read. Federation needs these success and delay assumptions for each replica; an uncooperative source has no bound.

Order-independent source evidence. Represent each source report by its source key, Card key, document digest, and check time. Merge reports by set union, grouped by Card key. Union is associative, commutative, and idempotent: regrouping, reordering, or repeating identical inputs gives the same evidence set. These are properties of retained evidence, not of a selected display body. First-wins body selection lacks the order property, as the worked example shows. If a source gives two versions, retain both reports unless a separate explicit source-version rule selects one. No merge operation turns multiple copies into independent confirmation.

Incomplete search is not an empty world. A result set from successful sources is a union over those sources only. No set operation over that union can infer the contents of a failed source without a further assumption. Reporting source failures therefore preserves a material part of the answer. Adding more sources may improve coverage, but repeated copies and unknown overlap prevent source count from measuring recall or truth.

VIII. Reproducible fixtures and control boundaries

A fixture consists of a clocked response map and a planned publisher state. The map takes URL, request headers, and time and returns status, headers, body, delay, and redirect target. The publisher state independently says which Cards are declared. For the mixed trace, both Catalogs list the Card initially; at the next tick only Catalog A removes it, while B delays beyond the fetch deadline. At the third tick B either returns its unchanged listing or removes it. Store both branches and their expected states.

Proposed resource settings are 1 MiB per document, ten seconds per request, five redirects, 64 discovery documents, eight publisher Catalogs, and a 60-second total check deadline. These are experimental settings, not measured optimal values. An interrupted or bounded-out path is unknown. Rotate which unresolved paths receive the next check budget so that a fixed prefix cannot starve older evidence. A rotation does not guarantee success under permanent failure.

The anonymous checker accepts only a Card URL and public hints. Reject extra input fields, send no private headers, and expose no private-store handle to it. Emit stage observations for Card read, shape, compatibility, association, and eligibility. The observation function can run without a public writer; a separate owner-checked transaction applies its result. A reproduction should save fixture definitions, schema versions, schedules, seeds, expected states, request logs, and transition logs.

An optional publisher-management proof uses a fresh challenge bound to one exact hostname or Card URL, a claimant, and an expiry. Hostname proof reads a DNS TXT token; Card proof reads a token at the exact URL without redirects. Missing tokens fail proof, while unavailable checks remain unknown. Such proof grants management of the stated scope only. It cannot make public metadata eligible, make a tool safe, or waive connection review. This is a proposed control boundary, not a requirement for publication.

IX. Finite model results

The Python experiment records 4,377 checks. Its artifact keeps the seeds, source hashes, results, and limits. The domains and methods below define its inputs. These checks exercise the abstract decision and merge rules. They do not test an HTTP client or a deployed index.

For each path count from one through five, enumerate every declaration vector and every availability vector. This gives $4^1 + 4^2 + 4^3 + 4^4 + 4^5 = 1364$ cases. A read is positive or negative when available, and unknown otherwise. Every case starts with a prior listing. The oracle enumerates every possible value of every unknown coordinate, computes publication in each completed world, and asks whether all agree. It does not use the decision function's branches. The proposed rule matches this oracle in every case. There are 358 cases whose compatible worlds disagree about publication.

The comparison uses the defined mixed-evidence and collapsed rules. A keep-prior control returns indexed on positive

evidence and unconfirmed otherwise. It makes no withdrawal decision in this experiment. No robots or `noindex` event occurs in the enumeration; direct `noindex` priority is a separate fixed check. A false withdrawal means that the policy withdraws while some declaration in the fixed truth vector is present. A clear withdrawal is one for which every compatible world is unpublished. There are five clear cases, one for each path count.

TABLE III
EXACT RESULTS OVER 1,364 FIXED-TRUTH CASES. COUNTS DESCRIBE THIS FINITE DOMAIN, NOT WEB FAILURE RATES.

Policy	False withdrawal	Unconfirmed	Missed clear withdrawal
Mixed evidence	244	62	0
Collapsed	301	0	0
Keep prior	0	363	5
Proposed	0	358	0

The result separates two easy ways to get zero false withdrawals. Keeping every prior listing also misses all five fully known withdrawals. The proposed rule keeps only the unresolved cases and acts on all five clear withdrawals. It leaves 57 cases visible even though the hidden truth vector has no declaration. Those cases have unresolved observations, so removing them as confirmed withdrawals would require extra information. This cost is a direct result of the observation model.

A separate seeded check uses seeds 0 through 29, with 60 cases per seed, for 1,800 cases. It selects one to five paths, fair binary declarations, and independently makes each path unavailable with probability 0.35. The mixed rule gives 176 false withdrawals; the proposed rule gives none under the fixed-truth assumption. These settings are declared synthetic inputs. They are not estimates of publisher behavior, and repeated finite cases do not add evidence about web frequency. The exhaustive comparison is the stronger check of the small state space; the seeded check tests the same property on another input sequence.

A merge fixture uses three sources, with three different bodies for one shared Card. All six source orders give one identical retained-evidence result. First-wins gives three different selected bodies. Another fixed trace executes the non-atomic read counterexample above. The publisher remains declared at every step, yet the rule withdraws after two negative reads. The test expects this failure outside the theorem's fixed-truth premise. Keeping this case in the artifact prevents a zero-error result on fixed states from being read as a guarantee for changing web documents.

There is no interval estimate for the exhaustive counts: they enumerate the stated domain rather than sample a population. The artifact does not test real robots handling, private-store separation, job transactions, or delayed network reads. Those require the next evaluation layer.

X. PROPOSED DEPLOYMENT AND EVALUATION

The evaluation should compare life-cycle decisions, not generated conformance suites or tool-call success. Build a controlled web fixture with independent origins for publishers, hosted Cards, unrelated Catalogs, and index sources. Serve real HTTP responses through a network adapter that records requests. Keep the fixture separate from the reference decision model. The model consumes a planned timeline of publisher intent, outages, and document changes, so it does not simply repeat implementation branches.

Compare the mention, collapsed, and mixed-evidence baselines with the proposed per-path rule. Pair each with first-wins search and with the disagreement-preserving merge. Use identical seeds, schedules, and response traces across policies. This separates the effect of publication aggregation from the effect of search merging.

Vary independent factors: one or several evidence paths; same-origin or hosted publication; stable or changed Card URL; valid omission or network failure; robots allow or deny; single-source or correlated index outage; and descriptive or sensitive endpoint changes. Include duplicated Catalogs and matching names at distinct URLs. Use small exhaustive traces for state rules, then seeded larger workloads for queue and search behavior. Report the workload generator and all seeds.

The main metrics are false admission without publisher evidence, false withdrawal during continued publication, delay from explicit withdrawal to hiding, time spent visible with an unconfirmed check, and source-failure reporting accuracy. Count forbidden private values in emitted requests and public outputs. Measure sensitive changes that reach a connection update without a review signal. These last two are boundary checks, not claims that all privacy or authorization risks have been tested.

For search, report unique Card coverage against the fixture's known set, duplicate collapse errors, version disagreements retained, and page restarts caused by changed source availability. Keep coverage separate from the responding-source fraction. For cost, measure documents, bytes, wall time, queue delay, and repeated work after ownership loss. Record timeouts as outcomes, not dropped samples.

Remove one component at a time: publisher association, per-path unknown state, explicit source issues, private-store separation, and final owner-token check. These ablations identify which mechanism causes each observed difference. For the owner-token case, pause an old worker, let another worker complete a later check, then resume the old worker. The expected observation is whether the old result changes the listing, not merely whether both workers return.

Use publisher timelines as the unit for paired comparisons. Repeated requests from one origin are dependent and should not be treated as independent trials. Report counts and per-timeline differences, with interval estimates from resampling whole timelines. Give exact trace results for deterministic state claims. Choose repetition counts after a separate pilot estimates timing spread, and keep pilot data out of final comparisons. Do not claim significance from a selected favorable failure rate.

Deployment should begin with a shadow index that computes all defined policies without changing public search. Review their disagreements against recorded publisher states and observations. A later limited rollout can expose partial-

source issues and conflict records. Clients must understand these states before relying on them. This staged plan is proposed work; no deployment is reported here.

XI. Limitations

Origin control is a limited foundation. Domain transfer, shared hosting mistakes, and compromised publisher systems can preserve apparently valid evidence while changing who stands behind it. Our method makes source claims easier to inspect; it does not solve publisher reputation, legal identity, or malicious behavior after connection.

Preserving uncertainty trades fewer false withdrawals for longer stale listings. A freshness limit moves that tradeoff but cannot remove it. Publisher-controlled `noindex` offers a clear direct route only when the crawler can read it. A robots block can prevent further observation. Host policy must explain why an entry is hidden rather than claim certainty about an unseen declaration.

Several sources do not form a trusted consensus. They may copy one another, use different publication rules, or deliberately keep withdrawn metadata. The proposed disagreement record exposes divergence but does not settle it. Cross-index withdrawal propagation and source accountability need separate operational agreements.

The referenced discovery drafts may change. Reproductions must freeze the document shapes and evidence rules used. Publication semantics affect saved entries and client interpretations, so changing them requires an explicit profile revision. This paper offers proofs, finite executable checks, and a concrete network evaluation method. The proofs rely on their stated observation assumptions. The checks do not prove product conformance, web-scale gains, or complete security.

XII. Conclusion

An open tool web can use publisher-controlled HTTPS documents for discovery without treating them as proof of trustworthiness. Stable Card identities, explicit evidence paths, separate private connections, and visible source failures give each observation a clear meaning. The defined baselines show how mixed evidence and first-wins merging lose information. The proposed rule makes every certain decision supported by a fixed partial observation, while leaving undecidable cases explicit. Permanent outages prevent a finite withdrawal bound, and separately read documents can miss a continuously present declaration. These limits define the scope of the result. Finite checks exercise the decision model; the stated network evaluation is still needed to measure its operating cost.

References

[1] T. Berners-Lee, R. Fielding, and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax," technical report RFC 3986, 2005. doi: 10.17487/RFC3986.

[2] M. Nottingham, "Well-Known Uniform Resource Identifiers (URIs)," technical report RFC 8615, 2019. doi: 10.17487/RFC8615.

[3] M. Nottingham, "Web Linking," technical report RFC 8288, 2017. doi: 10.17487/RFC8288.

[4] AI Catalog, "AI Catalog." Accessed: Oct. 27, 2026. [Online]. Available: https://ai-catalog.io/spec/

[5] Model Context Protocol, "Discovery." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/modelcontextprotocol/ext-server-card/blob/main/docs/discovery.md

[6] Model Context Protocol, "MCP Server Cards (Experimental Extension)." Accessed: Oct. 27, 2026. [Online]. Available: https://github.com/modelcontextprotocol/ext-server-card/blob/main/README.md

[7] Model Context Protocol, "Specification." Accessed: Oct. 27, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28

[8] M. Koster, G. Illyes, H. Zeller, and L. Sassman, "Robots Exclusion Protocol," technical report RFC 9309, 2022. doi: 10.17487/RFC9309.

[9] Google Search Central, "Robots Meta Tag, data-nosnippet, and X-Robots-Tag Specifications." Accessed: Oct. 27, 2026. [Online]. Available: https://developers.google.com/search/docs/crawling-indexing/robots-meta-tag

[10] R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," technical report RFC 9110, 2022. doi: 10.17487/RFC9110.

[11] P. Jones, G. Salgueiro, M. Jones, and J. Smarr, "WebFinger," technical report RFC 7033, 2013. doi: 10.17487/RFC7033.

[12] T. D. Chandra and S. Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996, doi: 10.1145/226643.226647.

[13] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance Semirings," in *Proceedings of the ACM Symposium on Principles of Database Systems*, 2007, pp. 31–40. doi: 10.1145/1265530.1265535.

[14] T. Imieliński and W. Lipski Jr., "Incomplete Information in Relational Databases," *Journal of the ACM*, vol. 31, no. 4, pp. 761–791, 1984, doi: 10.1145/1634.1886.